

Classification

Start by piecing together the code cells from the slides that use `tidymodels` to predict the `species` (Gentoo or Adelie) of penguins using the value of `body_mass_g`. Put them into an `.r` script or Quarto document.

The following exercises have you investigate the different components of the code and make variations.

1. To build an understanding of the four metrics used to evaluate model performance, write four `dplyr` (`filter()`, `summarize()`, `mutate()`, etc.) pipelines to calculate each metric manually from `ad_gen_preds`. Alongside each of the pipelines, write the English interpretation of each (e.g. “the proportion of ____ penguins that the model predicted to be ____”). Check your results against the values in `all_metrics`.
2. Using the model that performed the *worst*, build a plot to understand where it made the errors. This should be a scatterplot with the predictor along the x-axis, the species along the y-axis, and points colored by whether it is a true positive, true negative, false positive, or false negative.
3. Add two additional predictors to the recipe: `island` and `bill_length_mm`. Also add a third step: `step_dummy(all_nominal_predictors())`. What are the different values that `island` can take?
4. Repeat the prep and bake phases as in the last problem set to get a glimpse of what the training data would look like when all of the steps are applied. What happened to the `island` column?

5. Rerun your original three models with this modified recipe. How does the predictive performance of each model on the test set compare?

-
6. Install the `kernlab` package and read the help file for `data(spam)`. What is the natural response variable in this dataset? What do each of the predictors represent?

7. Using the `spam` dataset, create a classification model to predict whether an email is spam or not spam. You may use whichever predictors you like and any model class (with any parameters). Use a 70/30 train/test split and evaluate the model using the same four metrics as in the penguin example. What is the highest you were able to get your test accuracy?

APIs

Coming soon!